

hw3_sol

1 CS233 Data Structure Homework Assignment #3

Remarks:

- Delayed submission will get penalty in score:
 - 10%, 20% and 30% off for delay of 0 ~ 24 hours, 24 ~ 48 hours and 48 ~ 72 hours, respectively;
 - and delay longer than 72 hours will not be accepted.

1.1 1) 8.7

What are the minimum and maximum number of internal and external nodes in an improper binary tree with n nodes?

Sol) A tree that is one long path would have $n-1$ internal nodes and 1 external node. A tree that is proper (and thus odd n) would $n+1$ have $\frac{n-1}{2}$ internal nodes and $\frac{n+1}{2}$ external nodes.

1.2 2) 8.10

Give a direct implementation of the num children method within the class `BinaryTree`.

```
[ ]: def num_children(self, p):  
    count = 0  
    if self.left(p) is not None:  
        count += 1  
    if self.right(p) is not None:  
        count += 1  
    return count
```

1.3 3) 8.20

Draw a binary tree T that simultaneously satisfies the following: * Each internal node of T stores a single character. * A preorder traversal of T yields EXAMFUN. * An inorder traversal of T yields MAFXUEN.

[link](#)

1.4 4) 8.21

In what order are positions visited during a preorder traversal of the tree of Figure 8.8?

Sol) - / x + 3 1 3 + - 9 5 2 + x 3 - 7 4 6

1.5 5) 8.22

In what order are positions visited during a postorder traversal of the tree of Figure 8.8?

Sol) 3 1 + 3 x 9 5 - 2 + - 3 7 4 - x 6 + -

1.6 6) 8.30

The `build_expression` tree method of the `ExpressionTree` class requires input that is an iterable of string tokens. We used a convenient example, `'(((3+1)x4)/((9-5)+2))'`, in which each character is its own token, so that the string itself sufficed as input to build expression tree. In general, a string, such as `'(35 + 14)'`, must be explicitly tokenized into list `['(', '35', '+', '14', ')']` so as to ignore whitespace and to recognize multidigit numbers as a single token. Write a utility method, `tokenize(raw)`, that returns such a list of tokens for a raw string.

```
[ ]: def tokenize(raw):
    arr = []
    accu = ""
    for word in raw:
        if word in "[]{}()+-x*/^":
            if accu != "":
                arr.append(accu)
                accu = ""
            arr.append(word)
        else:
            accu = accu + word
    return arr
```

1.7 7) 8.34

For a tree T , let n_I denote the number of its internal nodes, and let n_E denote the number of its external nodes. Show that if every internal node in T has exactly 3 children, then $n_E = 2n_I + 1$.

Sol) We will show this using induction. For $n_I = 0$, then $n_E = 2n_I + 1 = 1$. This is obviously true. For $n_I = 1$, then $n_E = 2n_I + 1 = 2 + 1 = 3$. Again, this is obviously true from our problem definition. Now let us assume that the n_E equation holds true for $k < k$, i.e., for any $n_I = k < k$, $n_E = 2n_I + 1$.

1.8 8) 8.43

We can define a binary tree representation T' for an ordered general tree T as follows (see Figure 8.23): * For each position p of T , there is an associated position p' of T' . * If p is a leaf of T , then p' in T' does not have a left child; otherwise the left child of p' is q' , where q is the first child of p in T . * If p has a sibling q ordered immediately after it in T , then q' is the right child of p' in T' ; otherwise p' does not have a right child.

Given such a representation T' of a general ordered tree T , answer each of the following questions:

- a. Is a preorder traversal of T' equivalent to a preorder traversal of T ?

- b. Is a postorder traversal of T' equivalent to a postorder traversal of T ?
- c. Is an inorder traversal of T' equivalent to one of the standard traversals of T ? If so, which one?

Sol) a) yes

b) no

c) yes, postorder

1.9 9) 8.50

Design algorithms for the following operations for a binary tree T : * **preorder_next(p)**: Return the position visited after p in a preorder traversal of T (or None if p is the last node visited). * **inorder_next(p)**: Return the position visited after p in an inorder traversal of T (or None if p is the last node visited). * **postorder_next(p)**: Return the position visited after p in a postorder traversal of T (or None if p is the last node visited).

What are the worst-case running times of your algorithms?

Sol)

preorder_next(v):

```

    if v is internal then
        return v's left child
    else
        p = parent of v
        if v is left child of p then
            return right child of p
        else
            while v is not left child of p do
                v = p
                p = p.parent
            return right child of p

```

inorder_next(v):

```

    if v is internal then
        p = left child of v
        while p has left child do
            p = left child of p
        return p
    else
        p = parent of v
        if v is left child of p then
            return p
        else
            while v is not left child of p do
                v = p
                p = p.parent
            return p

```

postorder_next(v):

```

if v is internal then
    p = parent of v
    if v is right child of p then
        return p
    else
        v = right child of p
        while v is not external do
            v = left child of v
        return v
    else
        p = parent of v
        if v is left child of p then
            return right child of p
        else
            return p

```

The worst-case running times for these algorithms are $O(h)$ where h is the height of the tree T .

1.10 10) 8.56

The **indented parenthetic representation** of a tree T is a variation of the parenthetic representation of T (see Code Fragment 8.25) that uses indentation and line breaks as illustrated in Figure 8.24. Give an algorithm that prints this representation of a tree.

Sol)

```

indented_parenthetic_representation(T, v, indent):
    print out indent number of tabs
    if T is external then
        print v.element()
    else
        indent += 1
        print v.element() + "("
        for each child w in T.children(v) do
            indented_parenthetic_representation(T, w, indent)
        indent -= 1
        print out indent number of tabs

```

1.11 11) 9.3

What does each remove min call return within the following sequence of priority queue ADT methods: add(5,A), add(4,B), add(7,F), add(1,D), remove min(), add(3,J), add(6,L), remove min(), remove min(), add(8,G), remove min(), add(2,H), remove min(), remove min()?

Sol) (1,D), (3,J), (4,B), (5,A), (2,H), (6,L).

1.12 12) 9.7

Illustrate the execution of the selection-sort algorithm on the following input sequence: (22, 15, 36, 44, 10, 3, 9, 13, 29, 25).

Sol)

22	15	36	44	10	3	9	13	29	25
3	15	36	44	10	22	9	13	29	25
3	9	36	44	10	22	9	13	29	25
3	9	10	44	36	22	15	13	29	25
3	9	10	13	36	22	15	44	29	25
3	9	10	13	15	22	36	44	29	25
3	9	10	13	15	22	36	44	29	25
3	9	10	13	15	22	25	44	29	36
3	9	10	13	15	22	25	29	44	36
3	9	10	13	15	22	25	29	36	44

1.13 13) 9.8

Illustrate the execution of the insertion-sort algorithm on the input sequence of the previous problem.

Sol)

22	15	36	44	10	3	9	13	29	25
15	22	36	44	10	3	9	13	29	25
15	22	36	44	10	3	9	13	29	25
10	15	22	36	44	3	9	13	29	25
3	10	15	22	36	44	9	13	29	25
3	9	10	15	22	36	44	13	29	25
3	9	10	13	15	22	36	44	29	25
3	9	10	13	15	22	29	36	44	25
3	9	10	13	15	22	25	29	36	44

1.14 14) 9.10

At which positions of a heap might the third smallest key be stored?

Hint) Where might the second smallest key be?

Sol) For min heap, third smallest key can be stored at child of second smallest key or sibling of second smallest key.

1.15 15) 9.13

Illustrate the execution of the in-place heap-sort algorithm on the following input sequence: (2, 5, 16, 4, 10, 23, 39, 18, 26, 15).

Hint) Mimic the illustration style used in the book for insertion-sort and selection-sort.

Sol)

Heap Construction

[2]

[5 2]

```
[16 2 5]
[16 4 5 2]
[16 10 5 2 4]
[23 10 16 2 4 5]
[39 10 23 2 4 5 16]
[39 18 23 10 4 5 16 2]
[39 26 23 18 4 5 16 2 10]
[39 26 23 18 15 5 16 2 10 4]
```

Removing a maximum element from the heap

```
[39 26 23 18 15 5 16 2 10 4]
[26 18 23 10 15 5 16 2 4] [39]
[23 18 16 10 15 5 4 2] [26 39]
[18 15 16 10 2 5 4] [23 26 39]
[16 15 5 10 2 4] [18 23 26 39]
[15 10 5 4 2] [16 18 23 26 39]
[10 4 5 2] [15 16 18 23 26 39]
[5 4 2] [10 15 16 18 23 26 39]
[4 2] [5 10 15 16 18 23 26 39]
[2] [4 5 10 15 16 18 23 26 39]
[2 4 5 10 15 16 18 23 26 39]
```

1.16 16) 9.19

Bill claims that a preorder traversal of a heap will list its keys in nondecreasing order. Draw an example of a heap that proves him wrong.

Sol) Imagine the heap which is represented by the array list [1, 5, 2, 8, 9, 7, 6]. This heap will not produce keys in nondecreasing order when a preorder traversal is used.

1.17 17) 9.20

Hillary claims that a postorder traversal of a heap will list its keys in nonincreasing order. Draw an example of a heap that proves her wrong.

Sol) Imagine the heap which is represented by the array list [1, 5, 2, 8, 9, 7, 6]. This heap will not produce keys in nonincreasing order when a postorder traversal is used.

1.18 18) 9.32

Assume that we are using a linked representation of a complete binary tree T , and an extra reference to the last node of that tree. Show how to update the reference to the last node after operations add or remove min in $O(\log n)$ time, where n is the current number of nodes of T . Be sure and handle all possible cases, as illustrated in Figure 9.12.

```
[ ]: # Utility called just after insert has been called. It updates 'last'
      # reference to be an external node of a proper binary tree to expand.
      def find_insertion_position(self):
          if self.is_empty():
              z = self.root()
```

```

else:
    z = last # assumed reference to current last position
    while not self.is_root(z) and z == self.right(self.parent(z)):
        z = self.parent(z) # walk upward
    if not self.is_root(z):
        z = self.right(self.parent(z)) # then go to right sibling
    while not self.is_leaf(z) # and finally
        z = self.left(z) # find leftmost i

```

1.19 19) 9.45

Write a key function for nonnegative integers that determines order based on the number of 1's in each integer's binary expansion.

```

[ ]: def count_bits(k):
    numOnes = 0
    tmp = k
    while tmp != 0:
        bit = tmp & 1
        if bit == 1:
            numOnes += 1
        tmp = tmp >> 1
    return numOnes

```

1.20 20) 9.50

An online computer system for trading stocks needs to process orders of the form “buy 100 shares at x each” or “sell 100 shares at

y each.” A buy order for x can only be processed if there is an existing sell order with price y such that $y \geq x$. Likewise, a sell order for y can only be processed if there is an existing buy order with price x such that $y \leq x$. If a buy or sell order is entered but cannot be processed, it must wait for a future order that allows it to be processed. Describe a scheme that allows buy and sell orders to be entered in $O(\log n)$ time, independent of whether or not they can be immediately processed.

Sol) Use two priority queues. One is for the buyer and the other is for the seller. We need to compare a new order against the lowest uncompleted sell-order or the highest uncompleted buy-order. Therefore, in the seller's priority queue a lower price will get higher priority, while in the buyer's priority queue a higher price will get higher priority. Time complexity of the comparison is $O(1)$. Thus, the overall time complexity is $O(\log n) + O(\log n) + O(1) = O(\log n)$.

1.21 21) 10.8

What would be a good hash code for a vehicle identification number that is a string of numbers and letters of the form “9X9XX99X9XX999999”, where a “9” represents a digit and an “X” represents a letter?

Sol) Either polynomial hash codes or cyclic shift hash codes would be good.

1.22 22) 10.9

Draw the 11-entry hash table that results from using the hash function, $h(i) = (3i + 5) \bmod 11$, to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, and 5, assuming collisions are handled by chaining.

Sol)

0	1	2	3	4	5	6	7	8	9	10
13	94				44			12	16	20
	39				88			23	5	
					11					

1.23 23) 10.10

What is the result of the previous exercise, assuming collisions are handled by linear probing?

Sol)

0	1	2	3	4	5	6	7	8	9	10
13	94	39	16	5	44	88	11	12	23	20

1.24 24) 10.11

Show the result of Exercise R-10.9, assuming collisions are handled by quadratic probing, up to the point where the method fails.

Sol)

0	1	2	3	4	5	6	7	8	9	10
13	94	39	11		44	88	16	12	23	20

The probe sequence when inserting value 5 fails to ever find the sole remaining empty slot (at index 4).

1.25 25) 10.12

What is the result of Exercise R-10.9 when collisions are handled by double hashing using the secondary hash function $h'(k) = 7 - (k \bmod 7)$?

Sol)

0	1	2	3	4	5	6	7	8	9	10
13	94	23	88	39	44	11	5	12	16	20

1.26 26) 10.14

Show the result of rehashing the hash table shown in Figure 10.6 into a table of size 19 using the new hash function $h(k) = 3k \bmod 17$.

Sol)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
		12	18	41		36	25		54			38	10		90	28		

1.27 27) 10.22

What is the expected running time of the methods for maintaining a maxima set if we insert n pairs such that each pair has lower cost and performance than one before it? What is contained in the sorted map at the end of this series of operations? What if each pair had a lower cost and higher performance than the one before it?

Sol) For this problem, we assume we are using a skip list as the implementation for the sorted map. With a skip list, the expected running time for maintaining a maxima set if we insert n pairs such that each pair has lower price and performance than one before it is $O(n \log n)$, since each pair is inserted and does not remove any others. Thus, the i^{th} insertion runs in $O(\log i)$ time, and the total expected running time is $\sum n_i = 1 \log i$, which is $O(n \log n)$.

If, on the other hand, each pair had a lower price and higher performance than the one before it, then the overall running time is $O(n)$, because each new pair causes the removal of the pair that came before it. Therefore the size of the map is constant, and thus all its operations run in worst-case $O(1)$ time.

(If a `SortedTableMap` were used, with lower price/performance pairs, each pair is inserted at the beginning of the array list, and thus the insertion time dominates the search time, for overall $O(n^2)$ time. With lower price/higher performance pairs, the `SortedTableMap` would use $O(n)$ time overall.)

1.28 28) 10.28

On page 406 of Section 10.1.3, we give an implementation of the method `setdefault` as it might appear in the `MutableMapping` abstract base class. While that method accomplishes the goal in a general fashion, its efficiency is less than ideal. In particular, when the key is new, there will be a failed search due to the initial use of `__getitem__`, and then a subsequent insertion via `setitem`. For a concrete implementation, such as the `UnsortedTableMap`, this is twice the work because a complete scan of the table will take place during the failed `__getitem__`, and then another complete scan of the table takes place due to the implementation of `__setitem__`. A better solution is for the `UnsortedTableMap` class to override `setdefault` to provide a direct solution that performs a single search. Give such an implementation of `UnsortedTableMap.setdefault`.

```
[ ]: def setdefault(self, k, v):
      for item in self._table:
          if k == item._key:
              return item._value
      # did not find match for key
      self._table.append(self._Item(k,v))
```

```
return v
```

1.29 29) 10.37

Refactor our ProbeHashMap design so that the sequence of secondary probes for collision resolution can be more easily customized. Demonstrate your new framework by providing separate concrete subclasses for linear probing and quadratic probing.

Hint) Perhaps you might define a nonpublic method that generates probe indices.

```
[ ]: from collections import MutableMapping
from random import randrange

class MapBase(MutableMapping):
    """Our own abstract base class that includes a nonpublic Item class."""
    #----- nested Item class -----
    class _Item:
        """Lightweight composite to store key-value pairs as map items."""
        __slots__ = '_key', '_value'

        def __init__(self, k, v):
            self._key = k
            self._value = v

        def __eq__(self, other):
            return self._key == other._key # compare items based on their keys

        def __ne__(self, other):
            return not (self == other) # opposite of __eq__

        def __lt__(self, other):
            return self._key < other._key # compare items based on their keys

class HashMapBase(MapBase):
    """Abstract base class for map using hash-table with MAD compression."""

    def __init__(self, cap=11, p=109345121):
        """Create an empty hash-table map."""
        self._table = cap * [None]
        self._n = 0 # number of entries in the map
        self._prime = p # prime for MAD compression
        self._scale = 1 + randrange(p-1) # scale from 1 to p-1 for MAD
        self._shift = randrange(p) # shift from 0 to p-1 for MAD

    def _hash_function(self, k):
```

```

        return (hash(k)*self._scale + self._shift) % self._prime % len(self.
→_table)

    def __len__(self):
        return self._n

    def __getitem__(self, k):
        j = self._hash_function(k)
        return self._bucket_getitem(j, k) # may raise KeyError

    def __setitem__(self, k, v):
        j = self._hash_function(k)
        print(j, k, v)
        self._bucket_setitem(j, k, v) # subroutine maintains self._n
        if self._n > len(self._table) // 2: # keep load factor <= 0.5
            self._resize(2*len(self._table) - 1) # number 2^x - 1 is often
→prime

    def __delitem__(self, k):
        j = self._hash_function(k)
        self._bucket_delitem(j, k) # may raise KeyError
        self._n -= 1

    def _resize(self, c): # resize bucket array to capacity c
        old = list(self.items()) # use iteration to record existing items
        self._table = c * [None] # then reset table to desired capacity
        self._n = 0 # n recomputed during subsequent adds
        for (k,v) in old:
            self[k] = v # reinsert old key-value pair

class ProbeHashMap(HashMapBase):
    """Hash map implemented with linear probing for collision resolution."""
    _AVAIL = object() # sentinel marks locations of previous deletions

    def _is_available(self, j):
        """Return True if index j is available in table."""
        return self._table[j] is None or self._table[j] is ProbeHashMap._AVAIL

    def _find_slot(self, j, k):
        """Search for key k in bucket at index j.

        Return (success, index) tuple, described as follows:
        If match was found, success is True and index denotes its location.
        If no match found, success is False and index denotes first available
→slot.
        """

```

```

firstAvail = None
while True:
    if self._is_available(j):
        if firstAvail is None:
            firstAvail = j # mark this as first avail
        if self._table[j] is None:
            return (False, firstAvail) # search has failed
        elif k == self._table[j]._key:
            return (True, j) # found a match
        j = (j + 1) % len(self._table) # keep looking (cyclically)

def _bucket_getitem(self, j, k):
    found, s = self._find_slot(j, k)
    if not found:
        raise KeyError('Key Error: ' + repr(k))
    return self._table[s]._value

def _bucket_setitem(self, j, k, v):
    found, s = self._find_slot(j, k)
    if not found:
        self._table[s] = self._Item(k,v)
        self.n += 1
    else:
        self._table[s]._value = v

def _bucket_delitem(self, j, k):
    found, s = self._find_slot(j, k)
    if not found:
        raise KeyError('Key Error: ' + repr(k))
    self._table[s] = ProbeHashMap._AVAIL

def __iter__(self):
    for j in range(len(self._table)):
        if not self._is_available(j):
            yield self._table[j]._key

```

```

[ ]: class MyHashMap(ProbeHashMap):

    def _hashing(self, j, i):
        return (j+1) % len(self._table) # default hashing

    def _find_slot(self, j, k):
        """Search for key k in bucket at index j.

        Return (success, index) tuple, described as follows:
        If match was found, success is True and index denotes its location.

```

If no match found, success is False and index denotes first available slot.

```

"""
firstAvail = None
i = 1
while True:
    if self._is_available(j):
        if firstAvail is None:
            firstAvail = j # mark this as first avail
        if self._table[j] is None:
            return (False, firstAvail) # search has failed
    elif k == self._table[j]._key:
        return (True, j) # found a match
    # j = (j + 1) % len(self._table) # keep looking (cyclically)
    j = self._hashing(j, i)
    i += 1

```

```

class QuadProbMap(MyHashMap):

    def _hashing(self, j, i):
        return (j+1) % len(self._table) + i**2

```

1.30 30) 10.42

Suppose that each row of an $n \times n$ array A consists of 1's and 0's such that, in any row of A , all the 1's come before any 0's in that row. Assuming A is already in memory, describe a method running in $O(n \log n)$ time (not $O(n^2)$ time!) for counting the number of 1's in A .

Sol) To count the number of 1's in A , we can do a binary search on each row of A to determine the position of the last 1 in that row. Then we can simply sum up these values to obtain the total number of 1's in A . This takes $O(\log n)$ time to find the last 1 in each row. Done for each of the n rows, then this takes $O(n \log n)$ time.